

JAVA

Generics em Java

Classes Genéricas • Bounds • Wildcards • Type Erasure

Apostila completa do básico ao avançado, com exemplos práticos para consolidar Generics em Java.

CONTEÚDO DA APOSTILA

- 01 O Problema Antes dos Generics
- 02 Criando Classes e Métodos Genéricos
- 03 Limites Superiores (Bounds)
- 04 A Armadilha da Covariância
- 05 Wildcards, PECS e Type Erasure
- 06 Comparable e Comparator
- 07 Generics no Java Moderno

Sumário

- 01 O Problema Antes dos Generics**
De coleções sem tipo a um compilador que protege você

- 02 Criando Classes e Métodos Genéricos**
Parâmetros de tipo, Diamond Operator e Raw Types

- 03 Limites Superiores (Bounds)**
Restringindo tipos com extends e métodos estáticos genéricos

- 04 A Armadilha da Covariância**
Por que List não é List

- 05 Wildcards, PECS e Type Erasure**
Os três coringas e o apagamento de tipo

- 06 Comparable e Comparator**
Ordem natural e ordens alternativas

- 07 Generics no Java Moderno**
Streams, var, Records e o Repository Pattern

01 O Problema Antes dos Generics

Por que o Java precisou inventar um jeito de parametrizar tipos

Até a versão 5 do Java, lançada em 2004, as coleções da linguagem armazenavam tudo como `Object`. Isso quer dizer que qualquer tipo podia ser colocado dentro de qualquer coleção, sem nenhuma verificação em tempo de compilação. Para usar o valor de volta, era preciso fazer um cast manual — e se o tipo estivesse errado, o erro só aparecia em tempo de execução, na forma de uma `ClassCastException`, muitas vezes longe do ponto onde o dado errado foi inserido.

Os Generics resolveram exatamente esse problema: moveram a verificação de tipo do runtime para o compile-time. Com eles, o compilador sabe exatamente qual tipo uma coleção guarda, recusa inserções incompatíveis antes mesmo do programa rodar, e elimina a necessidade de casts manuais espalhados pelo código.

```
// ANTES dos Generics – frágil, sem checagem de tipo:
List pedidos = new ArrayList();
pedidos.add("Pedido #100");
pedidos.add(200); // mistura tipos, ninguém avisa
String p = (String) pedidos.get(0); // cast manual
String erro = (String) pedidos.get(1); // ClassCastException em runtime!

// COM Generics – seguro desde a compilação:
List<String> pedidos2 = new ArrayList<>();
pedidos2.add("Pedido #100");
// pedidos2.add(200); // erro de compilação – protegido!
String p2 = pedidos2.get(0); // sem cast, tipo garantido
```

Exemplo — o mesmo problema resolvido com e sem Generics

CUSTO ZERO EM RUNTIME

Generics são verificados apenas em tempo de compilação e depois removidos do bytecode, processo chamado *type erasure*. Ou seja: toda essa segurança extra não custa nenhum desempenho durante a execução do programa.

02 Criando Classes e Métodos Genéricos

Parâmetros de tipo, o Diamond Operator e os Raw Types

Declarando uma classe genérica

Para tornar uma classe genérica, acrescenta-se um parâmetro de tipo entre `< >` logo depois do nome da classe. O identificador mais tradicional é `T` (de *Type*), mas qualquer nome válido funciona. Dentro do corpo da classe, esse identificador é usado como se fosse um tipo de verdade — o compilador o substitui pelo tipo concreto sempre que a classe é usada.

Uma classe genérica funciona como um molde parametrizável: `Pacote<Livro>` e `Pacote<Eletronico>` são duas variações do mesmo molde, cada uma *type-safe* para seu próprio tipo, sem que seja preciso duplicar nenhuma linha de código.

```
// classe comum – presa a um único tipo:
class CaixaDeTexto { String conteudo; }

// classe genérica – funciona para qualquer tipo T:
class Pacote<T> {
    private T conteudo;
    public Pacote(T conteudo) { this.conteudo = conteudo; }
    public T getConteudo() { return conteudo; }
    public void setConteudo(T conteudo) { this.conteudo = conteudo; }
}

// uso – T é trocado pelo tipo concreto em cada instância:
Pacote<String> destino = new Pacote<>("Rua das Flores, 100");
Pacote<Integer> codigo = new Pacote<>(45230);
```

Exemplo — de uma classe fixa a uma classe genérica reutilizável

Convenções de nome e o Diamond Operator

A JDK segue uma convenção de nomes com uma única letra maiúscula: T (Type) para um tipo genérico qualquer, E (Element) para elementos de coleções, K e V (Key/Value) para mapas, e letras seguintes como S e U quando há mais de um parâmetro de tipo.

O Diamond Operator (<>), do Java 7, permite que o compilador infira o tipo a partir do lado esquerdo da atribuição, evitando repetição desnecessária. Já os Raw Types — usar a classe genérica sem nenhum parâmetro de tipo — existem apenas por compatibilidade com código anterior ao Java 5 e devem ser evitados em qualquer código novo.

```
// convenções de nomenclatura:
class Colecao<E> { List<E> itens; }
class Dicionario<K, V> { Map<K, V> entradas; }

// Diamond Operator – antes e depois do Java 7:
List<String> antes = new ArrayList<String>(); // verboso
List<String> depois = new ArrayList<>(); // preferido

// Raw Type – evite em código novo!
List bruta = new ArrayList();
bruta.add("texto"); bruta.add(10); // sem checagem nenhuma
```

Exemplo — convenções de nome, Diamond Operator e Raw Type

03 Limites Superiores (Bounds)

Restringindo o tipo genérico com extends e criando métodos estáticos genéricos

Por que restringir o tipo aceito

Sem nenhuma restrição, um parâmetro de tipo genérico aceita literalmente qualquer coisa — inclusive tipos que não fazem sentido nenhum para a lógica da classe. Um upper bound, escrito com extends,

resolve isso: `T extends Funcionario` garante que qualquer `T` usado ali é `Funcionario` ou um subtipo dele.

Essa restrição traz dois ganhos ao mesmo tempo: segurança, porque só tipos compatíveis podem ser usados, e acesso, porque dentro da classe genérica agora é possível chamar qualquer método de `Funcionario` em uma variável do tipo `T`, já que o compilador garante essa compatibilidade. Vale notar que `extends` aqui não significa herança de classe no sentido estrito — significa 'é do tipo ou implementa o tipo'.

```
interface Funcionario {
    String getNome();
    double getSalario();
}

// T precisa ser Funcionario ou subtipo – garante acesso a getNome():
class Equipe<T extends Funcionario> {
    private List<T> membros = new ArrayList<>();
    public void adicionar(T membro) { membros.add(membro); }
    public void listar() {
        membros.forEach(m -> System.out.println(
            m.getNome() + " – R$" + m.getSalario()));
    }
}

Equipe<Vendedor> equipeVendas = new Equipe<>(); // Vendedor implementa Funcionario
```

Exemplo — upper bound garantindo acesso a métodos do tipo restringido

Múltiplos bounds e métodos estáticos genéricos

É possível exigir que o tipo satisfaça mais de uma restrição ao mesmo tempo, separando-as com `&`. A regra é: no máximo uma classe, que precisa vir primeiro, seguida de zero ou mais interfaces.

Métodos `static` não podem usar o parâmetro de tipo da classe, já que não existe instância nenhuma envolvida. A saída é declarar um parâmetro de tipo próprio do método, antes do tipo de retorno — completamente independente do `T` da classe.

```
// múltiplos bounds – classe primeiro, interfaces depois:
class Repositorio<T extends EntidadeBase & Comparable<T> & Serializable> {
    private List<T> itens = new ArrayList<>();
    public void salvar(T item) { itens.add(item); }
    public List<T> ordenados() { return itens.stream().sorted().toList(); }
}

// método estático com parâmetro de tipo PRÓPRIO:
class Fabrica<T> {
    private T valor;
    static <E> Fabrica<E> criar(E elemento) {
        Fabrica<E> f = new Fabrica<>();
        f.valor = elemento;
        return f;
    }
}
```

Exemplo — bounds combinados e um método estático com tipo independente

04 A Armadilha da Covariância

Por que List não é subtipo de List

É tentador supor que, se Gerente é subtipo de Funcionario, então List<Gerente> também deveria poder ser atribuída a uma variável List<Funcionario>. Essa suposição está errada, e o Java bloqueia isso de propósito, para preservar a segurança de tipos da coleção.

Se essa atribuição fosse permitida, seria possível inserir um Funcionario comum dentro de uma lista que, em tempo de execução, é uma lista só de Gerente — corrompendo a integridade da coleção e provocando um ClassCastException mais tarde, ao tentar recuperar aquele elemento. O que continua funcionando é a herança de implementação: ArrayList<String> pode ser atribuída a List<String> porque ArrayList implementa List — desde que o parâmetro de tipo continue idêntico dos dois lados.

```
List<Gerente> gerentes = new ArrayList<>();

// NÃO compila – genéricos não são covariantes:
// List<Funcionario> lista = gerentes; // erro de compilação!

// se fosse permitido, isto quebraria a coleção em runtime:
// lista.add(new Vendedor()); // Vendedor não é Gerente!
// Gerente g = gerentes.get(0); // ClassCastException

// solução – wildcard para leitura segura:
List<? extends Funcionario> visualizacao = gerentes;
```

Exemplo — a covariância que não existe e o motivo por trás da regra

Situação	Compila?	Motivo
Gerente → Funcionario	Sim	Herança de classe comum (is-a)
ArrayList<T> → List<T>	Sim	ArrayList implementa List, mesmo T
List<Gerente> → List<Funcionario>	Não	Tipos genéricos não são covariantes
List<? extends Funcionario>	Sim (leitura)	Wildcard com upper bound resolve
List<? super Gerente>	Sim (escrita)	Wildcard com lower bound resolve

05 Wildcards, PECS e Type Erasure

Os três tipos de coringa, a regra PECS e o apagamento de tipo

Os três tipos de wildcard

Um wildcard (?) representa um tipo desconhecido dentro de um argumento de tipo genérico. Ele só aparece em argumentos de tipo — nunca na declaração do parâmetro de tipo de uma classe ou método. Existem três variantes: o coringa irrestrito <?>, que aceita qualquer tipo; o upper bound <? extends T>, ideal para ler dados; e o lower bound <? super T>, ideal para escrever dados.

```
// 1. irrestrito – aceita qualquer List, só para leitura genérica:
void imprimirTodos(List<?> lista) {
    for (Object item : lista) System.out.println(item);
}

// 2. upper bound – lê valores de qualquer subtipo de Number:
double somarValores(List<? extends Number> valores) {
    return valores.stream().mapToDouble(Number::doubleValue).sum();
}

// 3. lower bound – escreve Integer ou qualquer supertipo dele:
void adicionarCodigos(List<? super Integer> lista, int qtd) {
    for (int i = 1; i <= qtd; i++) lista.add(i);
}
```

Exemplo — os três tipos de wildcard em uso

A regra PECS e o apagamento de tipo

PECS — Producer Extends, Consumer Super — é um mnemônico simples: se a coleção produz dados para você ler, use ? extends T; se ela consome dados que você escreve, use ? super T. Uma coleção que faz as duas coisas ao mesmo tempo deve usar um parâmetro de tipo normal, sem wildcard.

Type erasure é o mecanismo pelo qual o compilador remove os Generics do bytecode depois de validar tudo. Um T sem bound vira Object; um T extends Number vira Number. Isso significa que, em tempo de execução, List<String> e List<Integer> são indistinguíveis.

```
// PECS na prática – copiando de uma origem para um destino:
<T> void copiar(List<? extends T> origem, // Producer: extends
List<? super T> destino) { // Consumer: super
    for (T item : origem) destino.add(item);
}

// type erasure – bytecode aproximado após a compilação:
// class Pacote<T> { T item; } vira class Pacote { Object item; }

// consequência – não é possível instanciar T diretamente:
// T obj = new T(); // erro de compilação

// solução – receber uma factory:
<T> T criar(java.util.function.Supplier<T> fabrica) {
    return fabrica.get();
}
```

Exemplo — PECS aplicado e as consequências do type erasure

06 Comparable e Comparator

Ordem natural, ordens alternativas e `Comparator.comparing()`

Comparable — a ordem natural de um tipo

`Comparable<T>` é uma interface genérica da própria JDK com um único método: `compareTo(T outro)`. Implementá-la define a ordem natural de um tipo — a ordem usada por padrão em `Collections.sort()`, `Arrays.sort()` e estruturas como `TreeSet`.

O contrato do `compareTo` é: retornar negativo se `this` é menor que o argumento, zero se são iguais, e positivo se `this` é maior. A boa prática é usar os métodos estáticos das wrapper classes, como `Integer.compare(a, b)`, em vez de subtração manual, que pode sofrer overflow.

```
class Pedido implements Comparable<Pedido> {
    private int numero;
    private String cliente;
    private double valor;

    @Override
    public int compareTo(Pedido outro) {
        return Integer.compare(this.numero, outro.numero); // ordem natural
    }

    // comparadores alternativos como campos estáticos:
    public static Comparator<Pedido> POR_CLIENTE =
        Comparator.comparing(p -> p.cliente);
    public static Comparator<Pedido> POR_VALOR_DESC =
        Comparator.comparingDouble((Pedido p) -> p.valor).reversed();
}

List<Pedido> pedidos = carregarPedidos();
Collections.sort(pedidos); // ordem natural (por número)
pedidos.sort(Pedido.POR_CLIENTE); // por nome do cliente
pedidos.sort(Pedido.POR_VALOR_DESC); // do maior valor para o menor
```

Exemplo — ordem natural com Comparable e ordens alternativas com Comparator

Característica	Comparable	Comparator
Onde é implementado	Na própria classe	Classe separada, lambda ou campo estático
Método principal	compareTo(T outro)	compare(T a, T b)
Quantidade de ordens	Uma única — a natural	Quantas forem necessárias
API de uso	Collections.sort(lista)	lista.sort(comparador)

07 Generics no Java Moderno

Streams, var, Records e o Repository Pattern

Generics sustentam praticamente tudo o que há de moderno no Java. A API de Streams, do Java 8, depende deles o tempo todo — `Stream<T>`, `Optional<T>`, `Function<T, R>` e `Predicate<T>` são todos genéricos. O `var`, do Java 10, infere automaticamente o tipo genérico a partir do lado direito da atribuição. E Records, do Java 16, podem ser genéricos com uma sintaxe bastante enxuta.

```
// Java 8 – Streams usando Generics o tempo inteiro:
Stream<String> nomes = clientes.stream()
    .filter(c -> c.getIdade() >= 18) // Predicate<Cliente>
    .map(Cliente::getNome); // Function<Cliente, String>

// Java 10 – var infere o tipo genérico:
var listaDeNomes = new ArrayList<String>(); // ArrayList<String> inferido

// Java 16 – Records genéricos:
record Par<A, B>(A primeiro, B segundo) { }
Par<String, Integer> item = new Par<>("CEP", 45230);
```

Exemplo — Generics no Streams, no var e em Records

O Repository Pattern genérico

Um dos usos mais comuns de Generics em aplicações reais é o Repository Pattern: uma interface genérica que define operações de persistência (buscar, salvar, listar, remover) parametrizadas pelo tipo da entidade e pelo tipo do identificador. O mesmo contrato serve para qualquer entidade do sistema, sem duplicar código.

```

interface Repositorio<T, ID> {
    Optional<T> buscarPorId(ID id);
    List<T> buscarTodos();
    void salvar(T entidade);
    void remover(ID id);
}

// implementação em memória, reaproveitável para qualquer T:
class RepositorioEmMemoria<T, ID> implements Repositorio<T, ID> {
    private final Map<ID, T> dados = new HashMap<>();
    private final Function<T, ID> extratorDeId;

    public RepositorioEmMemoria(Function<T, ID> extratorDeId) {
        this.extratorDeId = extratorDeId;
    }
    @Override public void salvar(T entidade) {
        dados.put(extratorDeId.apply(entidade), entidade);
    }
    @Override public Optional<T> buscarPorId(ID id) {
        return Optional.ofNullable(dados.get(id));
    }
    @Override public List<T> buscarTodos() { return new ArrayList<>(dados.values()); }
    @Override public void remover(ID id) { dados.remove(id); }
}

// mesmo código, tipos completamente diferentes:
var pedidosRepo = new RepositorioEmMemoria<Pedido, Integer>(Pedido::getNumero);
var clientesRepo = new RepositorioEmMemoria<Cliente, String>(Cliente::getCpf);

```

Exemplo — Repository Pattern genérico reutilizado para entidades diferentes

ERRO COMUM

Dois métodos sobrecarregados que diferem apenas no parâmetro genérico (`processar(List<String>)` e `processar(List<Integer>)`) colidem depois do type erasure e não compilam — para o bytecode, os dois métodos ficam idênticos.

Resumo Final

Generics	Eliminam casting manual e garantem segurança de tipo em compile-time, sem custo em runtime.
Upper bound	<code>extends</code> restringe o tipo e libera acesso aos métodos daquele bound.
Covariância	List não é subtipo de List — use wildcards para resolver isso.
PECS	Producer Extends, Consumer Super — a regra prática para escolher o wildcard certo.
Type erasure	Generics são removidos do bytecode; T vira Object ou o bound declarado.